Day 02 — DSA Notes (Java)

> **Topic: Arrays — Basics to Intermediate**
>
> Arrays are the most fundamental data structure. This day covers traversal, searching, aggregation, and two-pointer / nested-loop patterns on arrays.

Table of Contents

11	Find Missing Element	Maths formula: n*(n+1)/2
12	Minimum Sum of Two Elements	Nested loops (brute force)
13	Print Pairs with at Least One Prime	Nested loops + isPrime helper
14	Find Common Odd Elements	Nested loops + odd & equality check

---

Array Basics — Quick Reference

```
// Declare and initialise
int[] arr = new int[Size];

// Fill from input
for (int i = 0; i < arr.length; i++) {
    arr[i] = scanner.nextInt();
}

// Traverse
for (int i = 0; i < arr.length; i++) {
    // use arr[i]
}
```

> **Key points:**
>
> - Arrays are **0-indexed** → first element = `arr[0]`, last = `arr[arr.length - 1]`
> - `arr.length` gives the size (not `arr.length - 1`)
> - Default value for `int[]` is `0`

Section 1: Accumulation / Aggregation

Problems

001 — Sum of Array Elements

Pattern: Start with `sum = 0`, add each element.

```
public static int Add(int[] arr) {
    int sum = 0;
    for (int i = 0; i < arr.length; i++) {
        sum = sum + arr[i];
    }
    return sum;
}
```

> **Example:** arr = [1, 2, 3, 4, 5] → sum = **15**

---

010 — Average of Array Elements

Pattern: Sum all elements, divide by count. Use `float` to avoid integer division.

```
public static float Sum(int[] arr) {
    float sum = 0;
    for (int i = 0; i < arr.length; i++) {
        sum = sum + (float) arr[i];
    }
    return sum;
}

// In main:
float average = totalSum / N;
```

> **Why `float` cast?** In Java, `int / int` = integer (truncated). Cast to `float` first to get decimal result.

Example: arr = [1, 2, 3, 4] → sum = 10 → average = **2.5**

Section 2: Finding Max / Min

002 — Find Largest Element

Pattern: Initialise `max = Integer.MIN_VALUE`, update whenever a bigger element is found.

```
public static int largestNum(int[] arr) {
    int max = Integer.MIN_VALUE;
    for (int i = 0; i < arr.length; i++) {
        if (arr[i] > max) {
            max = arr[i];
        }
    }
    return max;
}
```

> **Why `Integer.MIN_VALUE`?** It is the smallest possible int (-2,147,483,648), so any real array element will be greater. Safe for arrays with negative numbers.

---

003 — Find Minimum Element

Pattern: Mirror of above — initialise `min = Integer.MAX_VALUE`.

```
public static int MinNum(int[] arr) {
    int min = Integer.MAX_VALUE;
    for (int i = 0; i < arr.length; i++) {
        if (arr[i] < min) {
            min = arr[i];
        }
```

```
    }
    return min;
}
```

> **Why `Integer.MAX_VALUE`?** It is the largest possible int (2,147,483,647), so any real element will be smaller.

Goal	Initial Value
Find MAX	`Integer.MIN_VALUE`
Find MIN	`Integer.MAX_VALUE`

---

004 — Index of Largest Element

Pattern: Track both `max` and `index` — update both together.

```
public static int largestNum(int[] arr) {
    int max = Integer.MIN_VALUE;
    int index = 0;
    for (int i = 0; i < arr.length; i++) {
        if (arr[i] > max) {
            max = arr[i];
            index = i;       // keep track of position
        }
    }
    return index;
}
```

> **Key insight:** Whenever you update `max`, also update `index = i`.
>
> **Example:** arr = [3, 7, 2, 9, 1] → max = 9 at index **3**

---

009 — Find Second Largest Element

Pattern: Two-pass approach — find `max` first, then find the largest

element that is **not** `max`.

```
public static int SecondlargestNum(int[] arr) {
    int max = Integer.MIN_VALUE;
    int max2 = Integer.MIN_VALUE;

    // Pass 1: find max
    for (int i = 0; i < arr.length; i++) {
        if (arr[i] > max) {
            max2 = max;
            max = arr[i];
        }
    }

    // Pass 2: find second largest (strictly less than ma
    for (int i = 0; i < arr.length; i++) {
        if (arr[i] > max2 && arr[i] != max) {
            max2 = arr[i];
        }
    }
    return max2;
}
```

> **Why two passes?** Pass 1 has an edge case — if the update `max2 = max` happens before `max` is truly the global max, `max2` may be wrong. Pass 2 corrects it.
>
> **Example:** arr = [3, 7, 7, 9, 1] → max = 9, second largest = **7**

Section 3: Searching

005 — Count Occurrences of K

Pattern: Simple counter — increment whenever `arr[i] == k`.

```
public static int Counting(int[] arr, int k) {
    int count = 0;
    for (int i = 0; i < arr.length; i++) {
        if (arr[i] == k) {
            count++;
        }
    }
    return count;
}
```

> **Example:** arr = [1, 2, 3, 2, 2], k = 2 → count = **3**

---

006 — Find Index of K (Linear Search)

Pattern: Return the index as soon as `k` is found. Return `-1` if not found.

```
public static int kIndex(int[] arr, int k) {
    for (int i = 0; i < arr.length; i++) {
        if (arr[i] == k) {
            return i;        // early exit
        }
    }
    return -1;               // not found sentinel value
}
```

> **Linear Search** — checks every element one by one.
>
> **Time Complexity:** O(n) in the worst case.
>
> **Returning -1** is the standard convention for "not found" in Java (used in String.indexOf(), Arrays.binarySearch(), etc.).

Section 4: The "Total Sum / Product" Trick

007 — Max and Min Sum of N-1 Elements

Key Insight:

> To get the sum of all elements *except* one, compute the **total sum** and subtract that one element.
>
> - **Max sum of N-1 elements** = total sum – **minimum** element
> - **Min sum of N-1 elements** = total sum – **maximum** element

```
int sum = 0;
for (int i = 0; i < arr.length; i++) {
    sum = sum + arr[i];
}

int maximumSum = sum - Min(arr);   // remove the smallest
int minimumSum = sum - larges(arr); // remove the largest
```

> **Example:** arr = [1, 2, 3, 4, 5] → total = 15
>
> - Max sum of 4 elements = 15 – 1 = **14**
> - Min sum of 4 elements = 15 – 5 = **10**

---

008 — Product of N-1 Elements (for each index)

Key Insight:

> Total product of all elements divided by `arr[i]` gives the product of all *other* elements.

```
public static int Product(int[] arr) {
    int product = 1;
    for (int i = 0; i < arr.length; i++) {
        product = product * arr[i];
    }
    return product;
}
```

```
// In main:
int p = Product(arr);
for (int i = 0; i < arr.length; i++) {
    System.out.println(p / arr[i]);   // product excludin
}
```

> **Example:** arr = [2, 3, 4] → total product = 24
>
> - Excluding index 0 (2): 24/2 = **12**
> - Excluding index 1 (3): 24/3 = **8**
> - Excluding index 2 (4): 24/4 = **6**
>
> **Warning:** This approach fails if any element is `0` (division by zero). A robust solution would require prefix/suffix product arrays.

Section 5: Maths-Based Array Problems

011 — Find Missing Element in 1 to N

Key Formula:

> Sum of first N natural numbers = `N * (N + 1) / 2`
>
> Missing element = **Expected sum** − **Actual sum**

```
public static int findMissingElement(int[] array) {
    int n = array.length + 1;                // original r
    int totalSum = n * (n + 1) / 2;         // Gauss formu
    int sum = 0;
    for (int i = 0; i < array.length; i++) {
        sum = sum + array[i];
    }
    return totalSum - sum;
}
```

> **Example:** Array has elements from 1–6 with one missing: [1, 2, 4, 5, 6]
>
> - n = 6, expected sum = 6*7/2 = **21**
> - actual sum = 18
> - missing = 21 − 18 = **3**

Section 6: Nested Loop (Two-Element) Problems

012 — Minimum Sum of Any Two Elements

Pattern: Check every pair (i, j) where j > i. Track the minimum sum seen.

```
public static int minSum(int[] arr) {
    int min = Integer.MAX_VALUE;
    for (int i = 0; i < arr.length - 1; i++) {
        for (int j = i + 1; j < arr.length; j++) {
            int sum = arr[i] + arr[j];
            if (min > sum) {
                min = sum;
            }
        }
    }
    return min;
}
```

> **Time Complexity:** O(n²) — every unique pair is checked once.
>
> **Why `j = i + 1`?** To avoid duplicate pairs and self-pairing (arr[i] + arr[i]).
>
> **Example:** arr = [4, 2, 1, 3] → pairs: (4,2)=6, (4,1)=5, (4,3)=7, (2,1)=3, (2,3)=5, (1,3)=4 → min = **3**

---

013 — Print Pairs Where at Least One is Prime

Pattern: Generate all pairs with nested loops. For each pair, check if either element is prime.

```
public static boolean isPrime(int n) {
    if (n < 2) return false;
    for (int i = 2; i * i <= n; i++) {
        if (n % i == 0) return false;
    }
    return true;
}

// In main:
for (int i = 0; i < arr.length - 1; i++) {
    for (int j = i + 1; j < arr.length; j++) {
        if (isPrime(arr[i]) || isPrime(arr[j])) {
            System.out.println(arr[i] + " " + arr[j]);
        }
    }
}
```

> **Note:** Reuses the `isPrime` helper from Day 01 — good practice for code reuse.
>
> **Example:** arr = [2, 4, 3] → pairs: (2,4) ✓ (2 is prime), (2,3) ✓ (both prime), (4,3) ✓ (3 is prime)

---

014 — Find Common Odd Elements in Two Arrays

Pattern: For each element in `arr1`, check if it is odd AND exists in `arr2`.

```
public static void findDublicate(int[] arr1, int[] arr2)
    boolean notFound = false;
```

```
    for (int i = 0; i < arr1.length; i++) {
        if (arr1[i] % 2 != 0) {                 // check i
            for (int j = 0; j < arr2.length; j++) {
                if (arr1[i] == arr2[j]) {       // check
                    System.out.print(arr1[i] + " ");
                    notFound = true;
                }
            }
        }
    }
    if (!notFound) {
        System.out.print("No common odd elements found.")
    }
}
```

> **Logic flow:**
>
> 1. Is `arr1[i]` odd? → If no, skip.
> 2. Is `arr1[i]` found in `arr2`? → If yes, print it.
>
> **Example:** arr1 = [1, 2, 3, 4], arr2 = [3, 5, 1, 6] → common odd = **1, 3**

Key Patterns Summary

Pattern	Code	Used In
Accumulator	`sum += arr[i]`	001, 007, 010, 011
Running max	`if (arr[i] > max) max = arr[i]`	002, 004, 007, 009
Running min	`if (arr[i] < min) min = arr[i]`	003, 007
Linear search	Loop + `== k`, return index or -1	005, 006
Total trick	`total - one_element`	007, 008

Gauss formula	`n*(n+1)/2`	011
Nested loops (pairs)	`i=0..n-1`, `j=i+1..n`	012, 013, 014
Helper function	`isPrime()`, `largestNum()`	007, 009, 013

Common Pitfalls

Mistake	Fix
Starting `max = 0` instead of `Integer.MIN_VALUE`	Fails for all-negative arrays
Starting `min = 0` instead of `Integer.MAX_VALUE`	Fails for all-positive arrays
`int avg = sum / n` (integer division)	Cast to `float` or `double` first
Division by zero in product trick	Handle `0` elements separately
Pairing `arr[i]` with itself in nested loops	Start inner loop at `j = i + 1`

Time Complexity Overview

Problem	Time Complexity	Reason
Sum / Max / Min / Search	O(n)	Single traversal
Second Largest	O(n)	Two passes (both O(n))

Min Sum of Two / Pairs	$O(n^2)$	Nested loops
Common Odd Elements	O(n1 * n2)	Nested loops
Missing Element (formula)	O(n)	Single sum pass

Concepts Checklist

- ☒ Array declaration, initialisation, and input
- ☒ Traversal with `for` loop using `arr.length`
- ☒ Accumulator pattern (sum, product)
- ☒ Running max/min with sentinel values (`Integer.MIN_VALUE`, `Integer.MAX_VALUE`)
- ☒ Tracking index alongside value
- ☒ Linear search and returning -1 for not found
- ☒ Two-variable approach for second largest
- ☒ Total sum / product trick to exclude one element
- ☒ Gauss formula for sum 1..N
- ☒ Nested loops for pair problems
- ☒ Reusing helper functions across problems
- ☒ float cast for accurate average
- ☒ Boolean flag for "found / not found" cases

Date: Day 02 of 30 Days Unstoppable Challenge